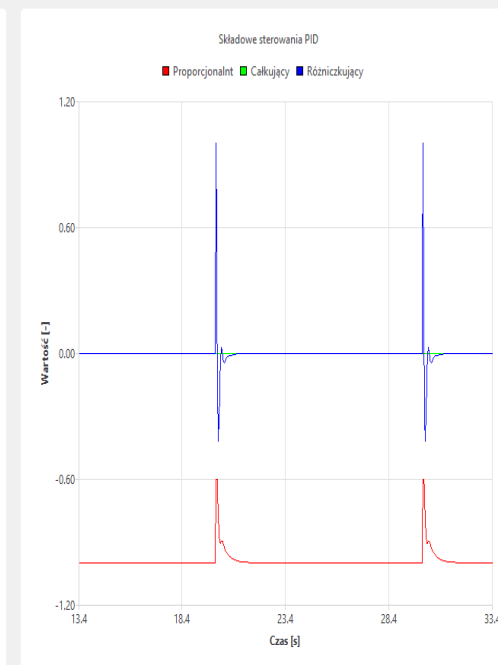
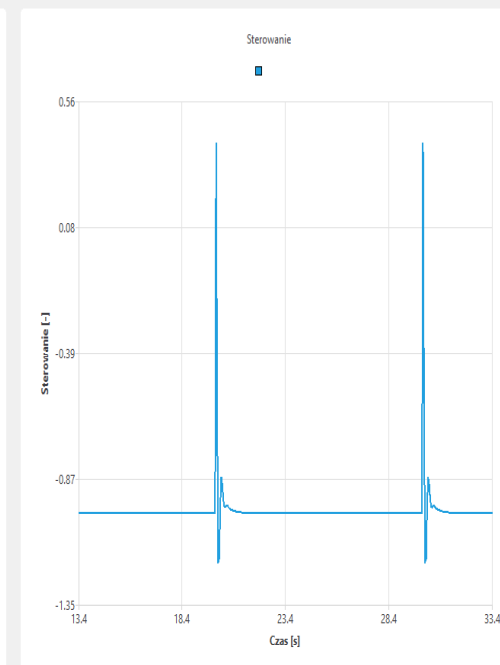
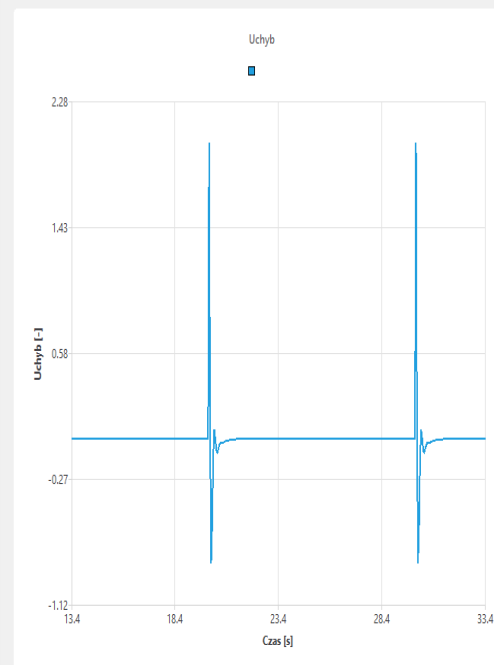
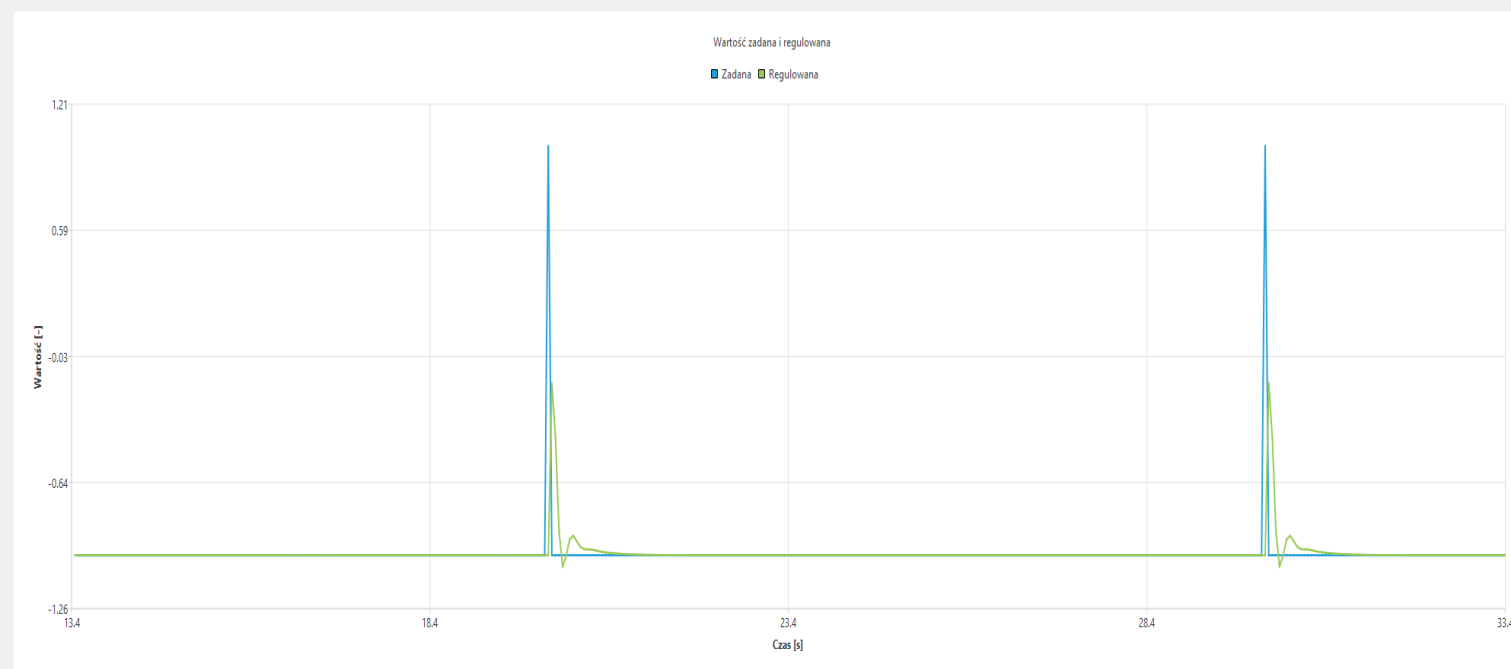


Prezentacja UAR

Sieci Komputerowe



Zapis
Odczyt

Start/Stop
Reset

Model ARX

Interval (ms)
50,00

Szerokość okna
20

☐ Regulator ON/OFF

Stewanie Uon
0,00

Histereza
0,00

☒ PID

Wzmocnienie
0,50

Stała całkowania
5,00

Stała różniczkowania
0,00

Reset pamięci całkującej

Reset pamięci różniczkującej

☐ Anti-windup

☐ Ograniczenia sterowania: Dolne -10,00 Górne 10,00

Tryb pracy całki: ☒ Stała w sumie ☐ Stała przed sumą

Dodaj sygnał
Usuń sygnał

☒ Sygnał prostokątny

Amplituda
1,00

Okres
10,00

Składowa stała
0,00

Wypełnienie
0,50

Poprzedni sygnał prostokątny:

☐ Sygnał sinusoidalny

Amplituda
0,00

Składowa stała
0,00

Okres
0,00

Wypełnienie
0,50

Poprzedni sygnał sinusoidalny:

Czas aktywacji
0,00

Zapis
Odczyt

Start/Stop
Reset

Model ARX

Interval (ms)
200,00

Szerokość okna
10

☐ Regulator ON/OFF

Stewanie Uon
0,00

Histereza
0,00

☐ PID

Wzmocnienie
0,00

Stała całkowania
0,00

Stała różniczkowania
0,00

Reset pamięci całkującej

Reset pamięci różniczkującej

☐ Anti-windup

☐ Ograniczenia sterowania: Dolne -10,00 Górne 10,00

Tryb pracy całki: ☒ Stała w sumie ☐ Stała przed sumą

Dodaj sygnał
Usuń sygnał

☐ Sygnał prostokątny

Amplituda
0,00

Okres
0,00

Składowa stała
0,00

Wypełnienie
0,50

Poprzedni sygnał prostokątny:

☐ Sygnał sinusoidalny

Amplituda
0,00

Składowa stała
0,00

Okres
0,00

Wypełnienie
0,50

Poprzedni sygnał sinusoidalny:

Czas aktywacji
0,00

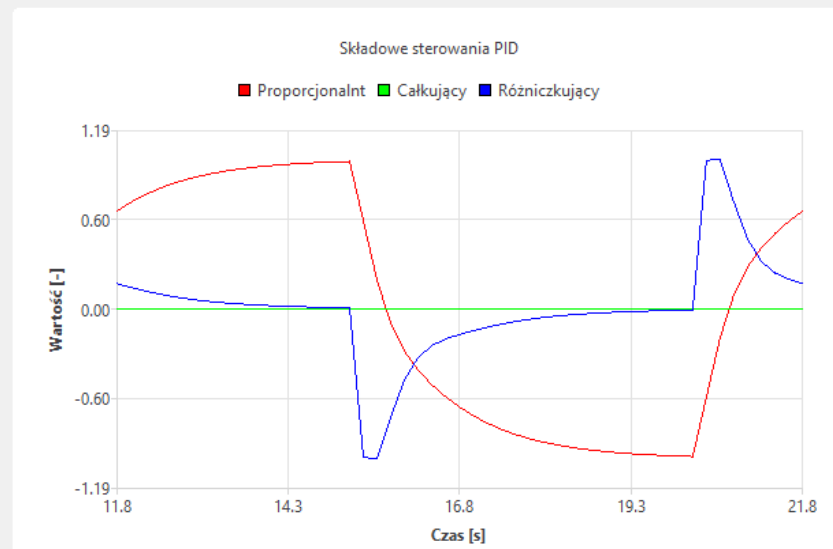
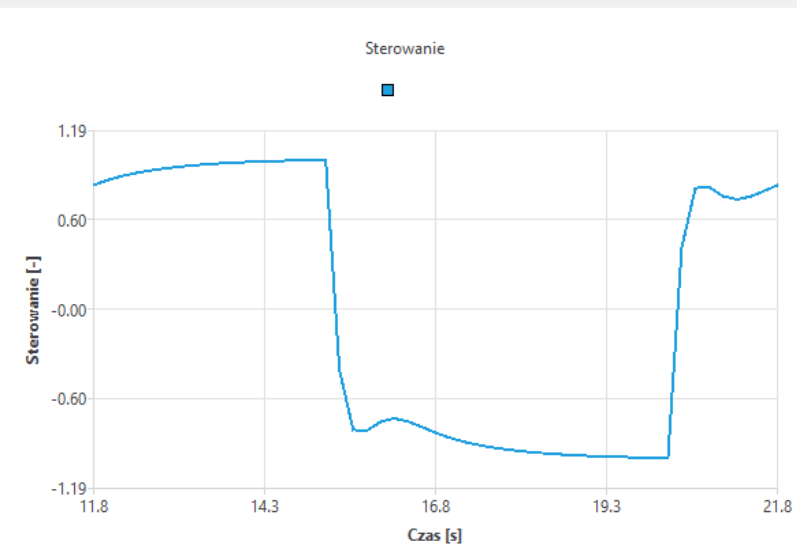
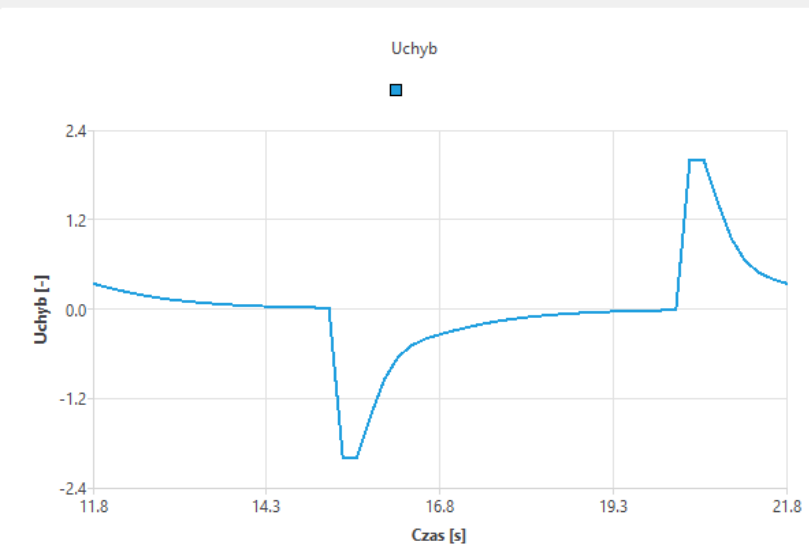
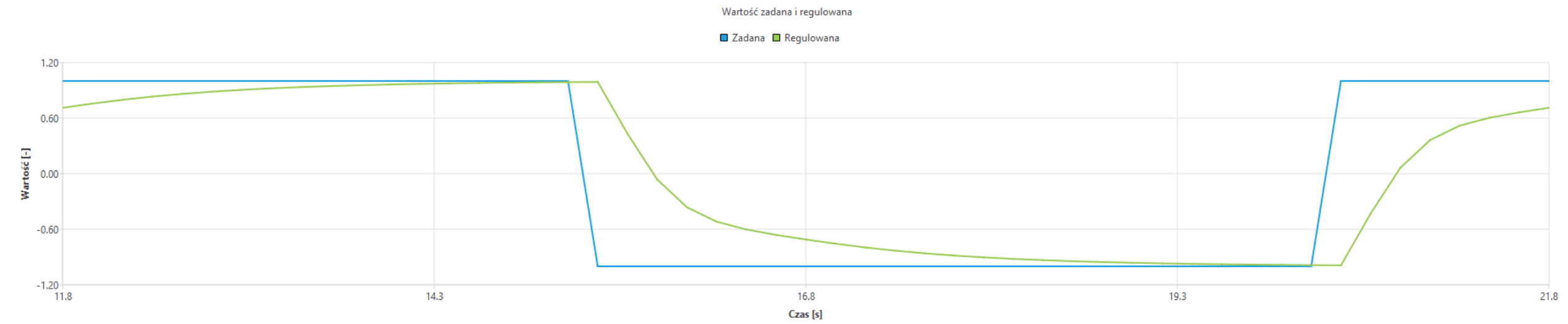
```
double Generator::oblicz_wartosc(double czas)
{
    if (czas < m_czas_aktywacji)
        return 0.0;

    if (m_typ == SINUS) {
        if (m_okres > 0) {
            return m_amplituda * std::sin(2 * M_PI * czas / m_okres) + m_offset;
        }
        return m_offset;
    } else if (m_typ == PROSTOKAT) {
        if (m_okres > 0) {
            double faza = std::fmod(czas, m_okres);
            double limit = m_okres * (m_wypelnienie / 100.0);
            if (faza < limit) {
                return m_amplituda + m_offset;
            } else {
                return -m_amplituda + m_offset;
            }
        }
        return m_offset;
    }
    return 0.0;
}
```

▶ prefix	
▶ suffix	
decimals	2
minimum	0.000000
maximum	1.000000
singleStep	0.100000
stepType	DefaultStepType
value	0.500000

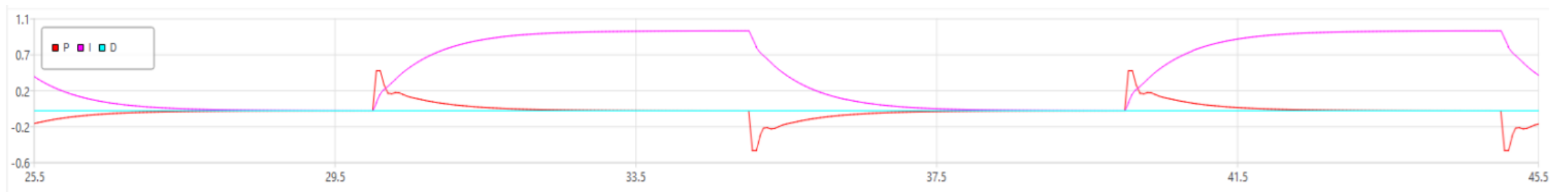
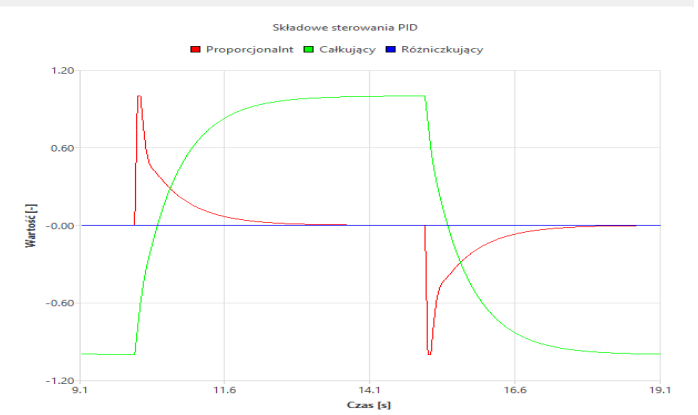
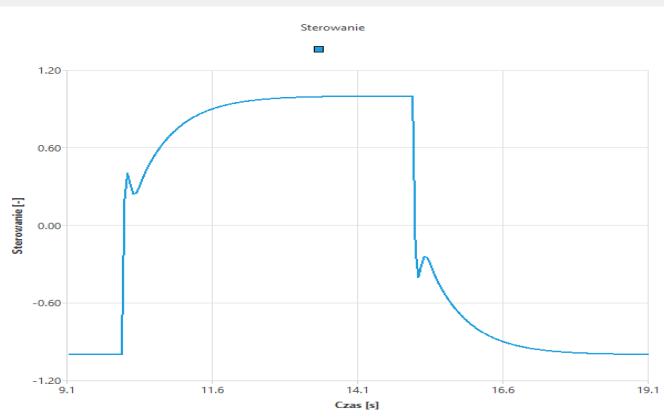
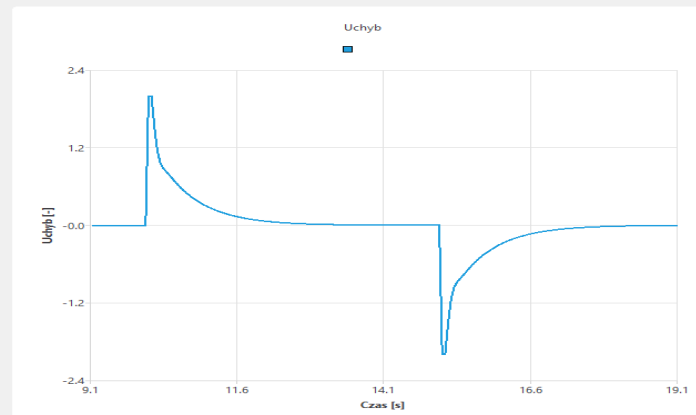
```
35
36 ui->fill_square_doubleSpinBox_2->setRange(0, 100);
37 is_running = false;
```

```
double limit = m_okres * m_wypelnienie;
```



```
//getery  
double get_wzmocnienie() const  
{  
    return m_ostatnia_skladowa_i;  
}  
double get_calka() const  
{  
    return m_ostatnia_skladowa_d;  
}  
double get_td() const  
{  
    return m_ostatnia_skladowa_p;  
}
```

```
void set_ograniczenia(double u_min, do  
void set_anti_windup(bool wl);  
//getery  
double get_wzmocnienie() const  
{  
    return m_ostatnia_skladowa_p;  
}  
double get_calka() const  
{  
    return m_ostatnia_skladowa_i;  
}  
double get_td() const  
{  
    return m_ostatnia_skladowa_d;  
}
```



Nawiązywanie połączenia

```
// ...  
void MainWindow::ustawStanPolaczony()  
{  
    ui->poleIP->setEnabled(false);  
    ui->polePort->setEnabled(false);  
    ui->btnPolacz->setEnabled(false);  
    ui->btnSerwer->setEnabled(false);  
    ui->btnRozlacz->setEnabled(true);  
    ui->lblStatus->setText("Status: POLACZONO");  
    ui->lblStatus->setStyleSheet("color: green; font-weight: bold;");  
    zablokujKontrolkiSymulacji(false);  
}  
  
void MainWindow::ustawStanRozlaczony()  
{  
    ui->poleIP->setEnabled(true);  
    ui->polePort->setEnabled(true);  
    ui->btnPolacz->setEnabled(true);  
    ui->btnSerwer->setEnabled(true);  
    ui->btnRozlacz->setEnabled(false);  
    ui->lblStatus->setText("Status: ROZLACZONO");  
    ui->lblStatus->setStyleSheet("color: red; font-weight: bold;");  
    zablokujKontrolkiSymulacji(true);  
}
```

```
void MainWindow::on_btnPolacz_clicked()  
{  
    klient->polaczZSerwerem(ui->poleIP->text(), ui->polePort->value());  
}  
  
void MainWindow::on_btnSerwer_clicked()  
{  
    if (serwer->uruchomSerwer(ui->polePort->value())) {  
        ui->btnSerwer->setEnabled(false);  
        ui->btnPolacz->setEnabled(false);  
        ui->lblStatus->setText("Status: OCZEKIWANIE...");  
        ui->lblStatus->setStyleSheet("color: orange; font-weight: bold;");  
    }  
}  
  
void MainWindow::on_btnRozlacz_clicked()  
{  
    klient->rozlaczZSerwerem();  
    serwer->zatrzymajSerwer();  
    ustawStanRozlaczony();  
}
```

```
//Tryby pracy
```

```
void MainWindow::setupRegulatorMode()
```

```
{
```

```
    isRegulatorMode = true;
```

```
    ui->ARX_model_button->setEnabled(false);
```

```
    ui->btnPolacz->setEnabled(false);
```

```
}
```

```
void MainWindow::setupObiektMode()
```

```
{
```

```
    isRegulatorMode = false;
```

```
    ui->PID_radioButton->setEnabled(false);
```

```
    ui->ONOFF_radioButton->setEnabled(false);
```

```
    ui->wzmocnienie_doubleSpinBox_3->setEnabled(false);
```

```
    ui->calka_doubleSpinBox_3->setEnabled(false);
```

```
    ui->rozniczka_doubleSpinBox_3->setEnabled(false);
```

```
    ui->histereza_doubleSpinBox_3->setEnabled(false);
```

```
    ui->antiwindup_checkBox_3->setEnabled(false);
```

```
    ui->checkBox_3->setEnabled(false);
```

```
    ui->sterowanie_doubleSpinBox_5->setEnabled(false);
```

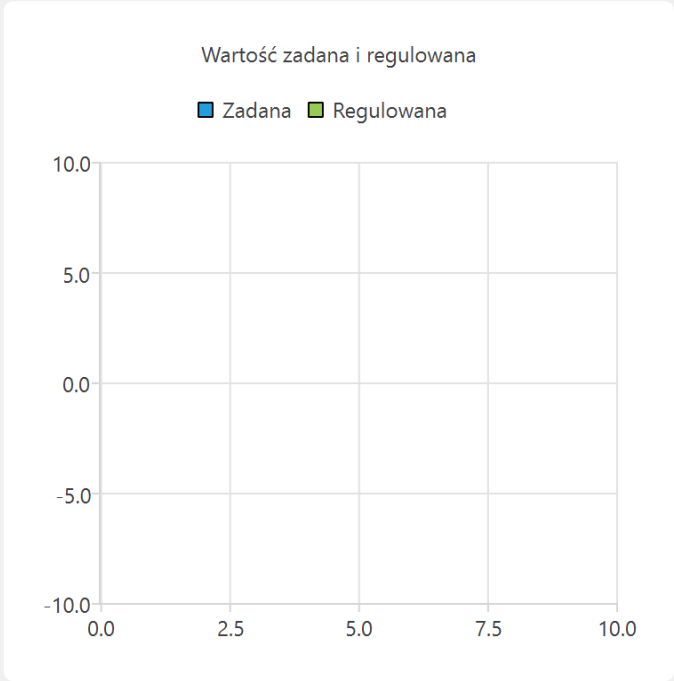
```
    ui->sterowanie_doubleSpinBox_6->setEnabled(false);
```

```
    ui->doubleSpinBox_6->setEnabled(false);
```

```
    ui->przed_radioButton->setEnabled(false);
```

```
    ui->w_sumie_radioButton->setEnabled(false);
```

```
    ui->sqrt_syg_radioButton->setEnabled(false);
```



Ustawienia połączenia TCP

Port: 5555

IP: 192.168.0.103

Rozłącz

Połącz (Klient)

Uruchom Serwer

Status: POLACZONO

Opóźnienie śr.: 0.0 ms Pakiety/s: 0 Straty: 0

Zapis

Odczyt

Start/Stop

Reset

Model ARX

Interwał (ms) 50,00

Szerokość okna 10

Regulator ON/OFF

Strewanie Uon 0,00

Histereza 0,00

PID

Wzmocnienie 0,50

Stała całkowania 5,00

Stała różniczkowania 0,20

Reset pamięci różniczkującej

Reset pamięci całkującej

Anti-windup

Ograniczenia sterowania: Dolne -10,00 Górne 10,00

Tryb pracy całki:

Stała w sumie

 Stała przed sumą

Dodaj sygnał

Usuń sygnał

Sygnał prostokątny

Sygnał sinusoidalny

Amplituda 1,00

Okres 3,00

Składowa stała 0,00

Wypełnienie 0,50

Poprzedni sygnał prostokątny:

Amplituda 1,00

Składowa stała 0,00

Okres 3,00

Poprzedni sygnał sinusoidalny:

Wartość zadana i regulowana

■ Zadana ■ Regulowana

Ustawienia połączenia TCP

Port: 5555

IP: 127.0.0.1

Rozłącz

Połącz (Klient)

Uruchom Serwer

Status: POLACZONO

Opóźnienie śr.: 0.0 ms Pakiety/s: 0 Straty: 0

Zapis

Odczyt

Start/Stop

Reset

Model ARX

Interwał (ms) 50,00

Szerokość okna 10

Regulator ON/OFF

Strewanie Uon 0,00

Histereza 0,00

PID

Wzmocnienie 0,50

Stała całkowania 5,00

Stała różniczkowania 0,20

Reset pamięci różniczkującej

Reset pamięci całkującej

Anti-windup

Ograniczenia sterowania: Dolne -10,00 Górne 10,00

Tryb pracy całki:

Stała w sumie

 Stała przed sumą

Dodaj sygnał

Usuń sygnał

Sygnał prostokątny

Sygnał sinusoidalny

Amplituda 1,00

Okres 3,00

Składowa stała 0,00

Wypełnienie 0,50

Poprzedni sygnał prostokątny:

Amplituda 1,00

Składowa stała 0,00

Okres 3,00

Poprzedni sygnał sinusoidalny:

Uchyb

■

Sterowanie

■

Składowe st...

■ ■

Sterowanie

■

Składowe st...

■ ■

Definicja typów pakietów oraz struktury danych

```
// Typy pakietów dzięki temu możemy odczytać jaki rodzaj pakietu otrzymaliśmy
const quint8 PKT_CONFIG      = 0x01;
const quint8 PKT_CONTROL     = 0x02;
const quint8 PKT_OUTPUT      = 0x03;
const quint8 PKT_COMMAND     = 0x04;

// Struktura pomocnicza do pakietów CONTROL tworzy gotowy do wysłania pakiet
struct ControlPacket {
    int id;
    double u;
    static QByteArray toJson(int id, double u) {
        QJsonObject obj;
        obj["id"] = id;
        obj["u"] = u;
        return QJsonDocument(obj).toJson();
    }
};

//Funkcja do odczytu z pakietu przesłanego tekstowo
static bool fromJson(const QByteArray &data, int &id, double &u) {
    QJsonObject obj = QJsonDocument::fromJson(data).object();
    if (obj.contains("id") && obj.contains("u")) {
        id = obj["id"].toInt();
        u = obj["u"].toDouble();
        return true;
    }
    return false;
}

};|
```

Wysyłanie pakietów oraz obliczanie opóźnień

```
//Timer symulacji dla regulatora
void MainWindow::onNetworkTimer()
{
    if (!isRegulatorMode) return;
    if (!klient->isConnected()) return;
    //pobieranie wartości zadanej
    double czas = kontroler->get_aktualny_czas();
    double setpoint = kontroler->get_generator()->oblicz_wartosc(czas);
    //obliczanie uchybu
    double error = setpoint - lastReceivedY;
    //obliczanie sterowania
    double u = 0.0;
    if (ui->PID_radioButton->isChecked()) {
        u = kontroler->get_uar()->get_pid()->symuluj(error);
    } else if (ui->ONOFF_radioButton->isChecked()) {
        u = kontroler->get_uar()->get_onoff()->symuluj(error);
    }

    lastSentU = u;
    //wysłanie pakietu CONTROL
    sendControl(u);
    //pomiar czasu wysłania pakietu
    pendingPackets[currentPacketId] = QDateTime::currentDateTime();
    stats->packetSent();
    //po 500ms braku odpowiedzi pakiet jest uznawany za stracony
    QTimer::singleShot(500, this, [this, id = currentPacketId]() {
        if (pendingPackets.contains(id)) {
            pendingPackets.remove(id);
        }
    });
}
```

Obsługa odebranych pakietów

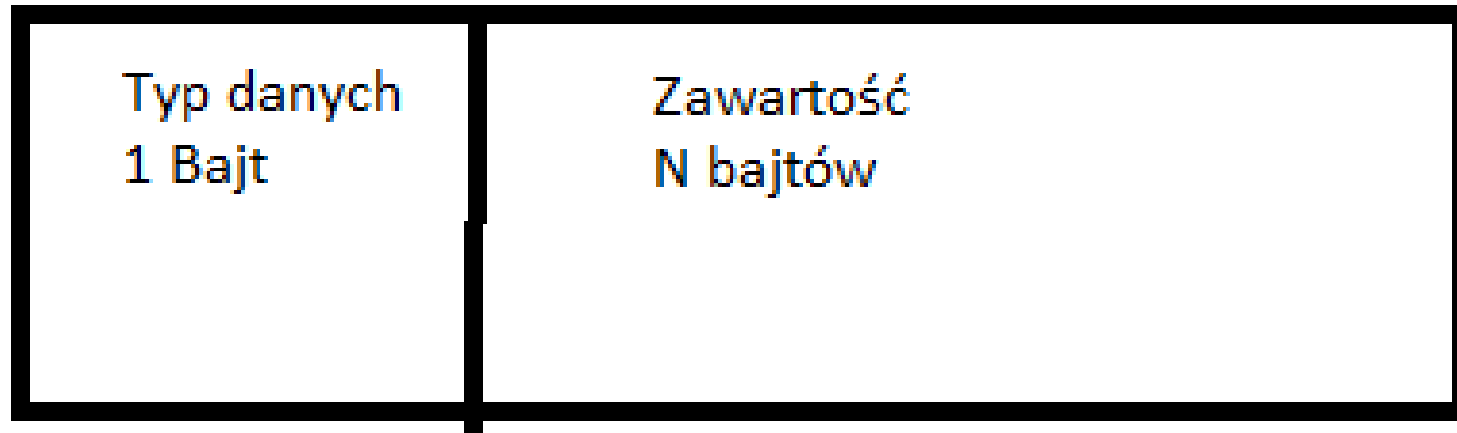
```
//Odbiór danych od serwera
void MainWindow::onServerDataReceived(quint8 typ, QByteArray data)
{
    if (typ == PKT_CONTROL) {
        int id; double u;
        if (ControlPacket::fromJson(data, id, u)) {
            //symulacja modelu arx
            double y = kontroler->get_uar()->get_model()->symuluj(u);
            //przygotowanie odpowiedzi
            QByteArray outData = OutputPacket::toJson(id, y);
            //wysyłanie OUTPUT
            serwer->wyslijDane(PKT_OUTPUT, outData);
            stats->packetSent();

            updateCharts(0.0, y, 0.0, u, 0.0, 0.0, 0.0);

            kontroler->symuluj_krok(time_step);
        }
    } else if (typ == PKT_CONFIG) {
        //aktualizacja modelu na podstawie otrzymanego pakietu
        ConfigData config = ConfigManager::deserializacja(data);
        if (kontroler && kontroler->get_uar() && kontroler->get_uar()->get_model()) {
            kontroler->get_uar()->get_model()->set_wspolczynniki_a(config.model.A);
            kontroler->get_uar()->get_model()->set_wspolczynniki_b(config.model.B);
            kontroler->get_uar()->get_model()->set_opoznienie(config.model.opoznienie);
        }
    }
}
```

Opis protokołu komunikacyjnego

Budowa ramki



Typy Pakietów

```
const quint8 PKT_CONFIG      = 0x01;  
const quint8 PKT_CONTROL    = 0x02;  
const quint8 PKT_OUTPUT     = 0x03;  
const quint8 PKT_COMMAND    = 0x04;  
const quint8 PKT_SYNC       = 0x05;  
const quint8 PKT_HEARTBEAT  = 0x06;
```

Struktura danych

```
QJsonObject obj = QJsonDocument::fromJson(data).object();
if (obj.contains("id") && obj.contains("t") && obj.contains("u")) {
    id = obj["id"].toInt();
    t = obj["t"].toDouble();
    u = obj["u"].toDouble();
    zadana = obj["zadana"].toDouble();
    uchyb = obj["uchyb"].toDouble();
    p = obj["p"].toDouble();
    i = obj["i"].toDouble();
    d = obj["d"].toDouble();
    return true;
}
```

```
double interval, bool isRunning;
QByteArray data;
QDataStream stream(&data, QIODevice::WriteOnly);
stream << clientId << serverId << clientT << serverT << interval << isRunning;
return data;
```

Serializacja/deserializacja

```
QByteArray ConfigManager::serializacja(const ConfigData &config)
{
    QJsonObject root;

    QJsonObject mainSettings;
    mainSettings["interwal_ms"] = config.main.interwal_ms;
    mainSettings["szerokosc_okna"] = config.main.szerokosc_okna;
    root["ustawienia_glowne"] = mainSettings;

    QJsonObject regulator;
    regulator["aktywny_regulator"] = config.regulator.typ;

    QJsonObject reg_pid;
    reg_pid["wzmocnienie"] = config.regulator.pid.k;
    reg_pid["stala_calkowania"] = config.regulator.pid.ti;
    reg_pid["stala_rozniczkowania"] = config.regulator.pid.td;
    regulator["pid"] = reg_pid;
    root["regulator"] = regulator;

    QJsonObject model_arx;
    QJsonArray a_arr, b_arr;
    for (double v : config.model.A) a_arr.append(v);
    for (double v : config.model.B) b_arr.append(v);

    model_arx["wspolczynniki_a"] = a_arr;
    model_arx["wspolczynniki_b"] = b_arr;
    model_arx["opoznienie_transportowe"] = config.model.opoznienie;
    root["model_arx"] = model_arx;

    QJsonDocument doc(root);
    return doc.toJson(QJsonDocument::Compact);
}
```

```
ConfigData ConfigManager::deserializacja(const QByteArray &dane)
{
    ConfigData config;

    config.main.interwal_ms = 100.0;
    config.main.szerokosc_okna = 10;

    QJsonDocument doc = QJsonDocument::fromJson(dane);
    if (!doc.isObject()) return config;

    QJsonObject root = doc.object();

    if (root.contains("ustawienia_glowne")) {
        QJsonObject m = root["ustawienia_glowne"].toObject();
        config.main.interwal_ms = m["interwal_ms"].toDouble();
        config.main.szerokosc_okna = m["szerokosc_okna"].toInt();
    }

    if (root.contains("regulator")) {
        QJsonObject r = root["regulator"].toObject();

        if (r.contains("pid")) {
            QJsonObject pid = r["pid"].toObject();
            config.regulator.pid.k = pid["wzmocnienie"].toDouble();
            config.regulator.pid.ti = pid["stala_calkowania"].toDouble();
            config.regulator.pid.td = pid["stala_rozniczkowania"].toDouble();
        }
    }

    if (root.contains("model_arx")) {
        QJsonObject arx = root["model_arx"].toObject();
        QJsonArray a = arx["wspolczynniki_a"].toArray();
        QJsonArray b = arx["wspolczynniki_b"].toArray();
        for (const auto &v : a) config.model.A.push_back(v.toDouble());
        for (const auto &v : b) config.model.B.push_back(v.toDouble());
        config.model.opoznienie = arx["opoznienie_transportowe"].toInt();
    }
}
```